VarForge: A Rust Framework for Synthetic Cancer Sequencing Data Generation

v0.2.0

Trent Zeng

April 2026

Abstract

Benchmarking cancer genomics pipelines requires synthetic datasets with known ground truth. No single tool integrates de-novo read generation, somatic variant spike-in, tumour modelling, UMI/duplex simulation, cfDNA fragmentation, library artefact injection, and platform-aware sequencing error models. VarForge is a pure-Rust framework that unifies these capabilities under a composable YAML configuration, achieving ∼7,000 read pairs/s on a 12-thread workstation with memory bounded by channel depth rather than dataset size. No other published tool generates genome-wide UMI-tagged reads with configurable family sizes and duplex strand pairing, or combines cfDNA fragmentation with somatic variant spike-in. Variant allele frequencies are assigned by per-read Bernoulli sampling rather than deterministic spike-in, eliminating the sensitivity optimism introduced by exact read counts. A four-scenario downstream benchmark with Mutect2 confirms that VarForge output exposes real caller behaviour across purity, coverage, and artefact conditions.

Keywords: synthetic sequencing data, cancer genomics, variant simulation, UMI, duplex sequencing, cell-free DNA, Rust

1 Introduction

Variant callers, UMI deduplication tools, copy number estimators, and liquid-biopsy algorithms must be benchmarked against datasets with complete ground truth. Real patient data cannot provide this. Synthetic simulation fills the gap.

Cancer sequencing introduces challenges that generic read simulators do not address. Tumour samples are mixtures of cancer and normal cells, parametrised by purity and clonal architecture. Allele frequencies are heterogeneous due to sub-clonal evolution. Copy number alterations affect both local coverage and effective VAF. Library preparation introduces artefacts specific to FFPE tissue and oxidative damage. Cell-free DNA from liquid biopsy has a distinct fragment length distribution that read simulators do not model.

No single existing tool covers this space. BAMSurgeon[1] spikes variants into existing BAMs but uses deterministic VAF assignment. ART[2] generates reads de novo but lacks variant injection. NEAT[3] supports basic SNV and indel injection but lacks tumour models, UMI simulation, and cfDNA fragmentation. ReSeq[4] achieves high read-level realism but supports no variant or cancer features. No genome-wide UMI or duplex simulator exists. No tool combines cfDNA fragmentation with somatic variant spike-in.

VarForge addresses these gaps in a single binary. The contributions are:

(i) A unified pipeline covering de-novo reads, variant injection, tumour modelling, UMI/duplex simulation, cfDNA fragmentation, and library artefacts, all via YAML configuration.
(ii) The first genome-wide UMI and duplex sequencing simulator.
(iii) The first in-silico cfDNA generator with somatic variant support.
(iv) Stochastic VAF sampling via per-read Bernoulli trials.
(v) A streaming Rust implementation with bounded memory.

2 Background

Three concepts are load-bearing for the method sections that follow.

Variant allele frequency (VAF). The fraction of sequencing reads at a locus that carry the alternate allele. In a tumour sample, VAF depends on tumour purity, ploidy, cancer cell fraction, and copy number. A heterozygous SNV in a pure diploid tumour has an expected VAF of 0.50. At 60% purity, the same variant has an expected VAF of 0.30.

Unique molecular identifiers (UMIs). Short random sequences (typically 4 to 14 bp) ligated to each

Table 1: Feature comparison of sequencing simulators. ✓ = supported; ∘ = partial; × = unsupported.

Tool	De-novo reads	Stoch. VAF	UMI/ Duplex	cfDNA	FFPE/ OxoG	CNV	Single binary
VarForge	✓	✓	✓	✓	✓	✓	✓
BAMSurgeon	×	×	×	×	×	∘	×
ART	✓	×	×	×	×	×	✓
NEAT	✓	×	×	×	×	×	×
stochasticSim	×	✓	×	×	✓	×	×
ReSeq	✓	×	×	×	×	×	×
VarSim	∘	×	×	×	×	✓	×
MOV&RSim	✓	×	×	×	∘	✓	×

DNA molecule before PCR amplification. After sequencing, reads sharing the same UMI and alignment position are grouped into families originating from one template molecule. Consensus calling across a family suppresses PCR and sequencing errors. Duplex sequencing extends this by tagging both strands of a double-stranded molecule independently, requiring agreement between strand-specific consensus reads before reporting a variant.

Cell-free DNA (cfDNA). Short DNA fragments circulating in blood plasma, released by apoptosis and necrosis. Normal cfDNA has a characteristic fragment length distribution dominated by a mono-nucleosomal peak at ∼167 bp (one nucleosome plus linker), with sub-nucleosomal and di-nucleosomal components. Tumour-derived cfDNA (ctDNA) fragments are shorter on average, peaking at ∼143–147 bp. This size difference is exploited by fragment-based liquid biopsy tools.

3 Related Work

Table 1 summarises the feature landscape. Tools fall into three categories: BAM editors (BAMSurgeon, SomatoSim[5], stochasticSim[6]), de-novo read generators (ART, NEAT, ReSeq, NGSNGS[7]), and cancer-evolution simulators (VarSim[8], PSiTE[9], MOV&RSim[10], GENOMICON-Seq[11]).

Two gaps stand out. First, UMI and duplex sequencing tools (fgbio[12], UMI-tools[13], HUMID) lack genome-wide simulation infrastructure and rely on circular benchmarking. Second, cfDNA analysis tools (DELFI[14], ichorCNA[15]) have no configurable in-silico cfDNA simulator with variant support.

A further limitation pervades BAM editors: deterministic spike-in. When BAMSurgeon injects a variant at 10% VAF into 30x data, it modifies exactly 3 reads. Real sequencing follows a binomial sampling process. Deterministic data overestimates caller sensitivity[6].

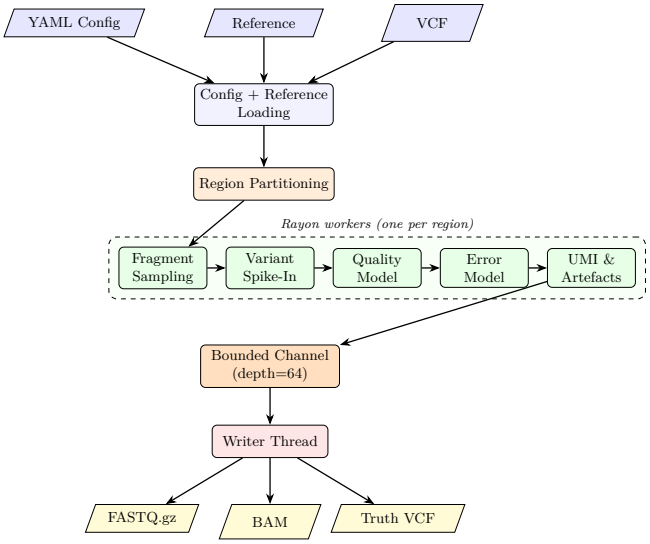

Figure 1: VarForge processing pipeline. Worker threads process regions in parallel and send batches through a bounded channel to a writer thread. Memory is bounded by channel depth, not dataset size.

4 Method

4.1 Architecture

Figure 1 shows the pipeline. A YAML configuration drives all parameters. The reference genome is loaded and shared via `Arc`. Variants come from VCF or stochastic generation. The genome is partitioned into fixed-size chunks (default 1 Mbp), each processed by a Rayon[16] worker thread. If a BED file of target intervals is provided, chunks are clipped to the target boundaries, restricting simulation to the specified regions. Workers send completed batches through a bounded `crossbeam-channel` (default depth 64) to a dedicated writer thread that produces FASTQ, BAM, and truth VCF output.

4.2 Read generation

Fragment lengths follow $\mathcal{N}(\mu, \sigma^2)$ for standard libraries or a cfDNA mixture model (Section 4.7). Start positions are drawn uniformly within each region. Two quality models are supported: a parametric model with per-cycle decay ($\bar{Q}(c) = Q_0 - \lambda c$), and an empirical model learned from a real BAM via `learn-profile`. The `learn-profile` subcommand extracts per-cycle quality distributions, per-cycle error rates, and indel rates from an existing BAM, enabling data-driven error profiles for any platform.

4.3 Stochastic VAF sampling

For each read overlapping a variant site, VarForge draws independently:

$$X_i \sim \text{Bernoulli}(\text{VAF}_{\text{eff}}) \tag{1}$$

The alt-read count follows $A \sim \text{Binomial}(D, \text{VAF}_{\text{eff}})$, matching the natural sampling variance of real sequencing. BAMSurgeon uses deterministic $A = \lfloor D \cdot \text{VAF} \rfloor$.

4.4 Variant spike-in

SNVs replace reference bases. Indels modify sequence and CIGAR with local re-alignment. MNVs are applied atomically to preserve cis-phasing. Structural variants (deletions, insertions, inversions, tandem duplications, translocations) produce split reads and discordant pairs with correct CIGAR strings, SA tags, and mate coordinates. Copy number variants scale read depth by $(p \cdot \text{CN}_{\text{tumour}} + (1-p) \cdot 2)/2$ and adjust effective VAF accordingly.

4.5 Tumour modelling

Heterogeneity is a directed tree of clones, each with a cancer cell fraction (CCF). The effective VAF integrates purity, ploidy, CCF, and copy number:

$$\text{VAF}_{\text{eff}} = \frac{\text{CCF} \times m \times p}{p \cdot \text{CN}_{\text{tumour}} + (1-p) \cdot \text{CN}_{\text{normal}}} \tag{2}$$

4.6 UMI and duplex simulation

Each molecule receives a UMI barcode of configurable length. Family sizes follow a log-normal distribution reflecting real PCR kinetics. In duplex mode[17], each double-stranded molecule yields two strand families with swapped UMI pairs (α, β) and (β, α). A configurable duplex conversion rate (`umi.duplex_conversion_rate`) controls the probability that a molecule produces both AB and BA strand families; the default is 1.0, and realistic Twist data uses ∼0.90. True mutations appear on both strands. Artefacts are strand-specific, reproducing the error suppression that makes duplex sequencing valuable.

When `umi.inline` is true, the UMI sequence and an optional spacer are prepended to R1 and R2 in both FASTQ and BAM output. BAM records receive a leading soft-clip CIGAR op covering the prefix. This format is compatible with downstream tools that expect inline UMIs (e.g. fgbio `ExtractUmisFromBam`, UMI-tools `extract`). A per-base UMI sequencing error rate (`umi.error_rate`) injects substitution errors into the inline UMI bases of each PCR copy, modelling the sequencing noise that UMI grouping algorithms must tolerate.

4.7 Cell-free DNA fragmentation

Fragment lengths follow a three-component Gaussian mixture:

$$P(L) = f_{\text{ct}} \cdot \mathcal{N}(L;\, 143,\, 15^2) + (1 - f_{\text{ct}}) \left[w_1 \cdot \mathcal{N}(L;\, \mu_{\text{mono}},\, \sigma^2_{\text{mono}}) + (1 - \right. \tag{3}$$

where f_{ct} is the ctDNA fraction and w_1 is the mono-nucleosomal weight. The three components represent ctDNA (∼143 bp), mono-nucleosomal (∼167 bp), and di-nucleosomal (∼334 bp) populations. A 10 bp sinusoidal periodicity matching the DNA helical repeat is applied to all sampled sizes[18]. Tumour-derived fragments are shorter (∼143 bp peak)[19], enabling fragment-based tumour fraction estimation by tools such as DELFI.

4.8 Library artefacts

FFPE deamination: C>T transitions at a configurable rate, preferentially at fragment ends. Oxidative damage: G>T transversions with strand-specific bias. PCR errors: per-cycle substitution errors propagated through family genealogies. GC bias: fragment-level coverage modulation via a Gaussian curve centred at 50% GC content, with a configurable severity parameter.

4.9 Sequencing error model

An optional layered error model (`quality.sequencing_errors`) replaces quality-score-driven substitutions with a five-pass `ErrorOrchestrator`. The passes are: (1) cycle-position errors, where the error rate rises with sequencing cycle; (2) k-mer context errors, applying sequence-context-dependent mismatch rates; (3) sequencing indels, inserting or deleting bases at configurable rates; (4) strand bias, modulating error rates between forward and reverse reads; and (5) phasing bursts, injecting correlated errors that mimic polymerase slippage events. The model is configured through 14 fields in the `quality.sequencing_errors` YAML block. Three built-in platform presets (`illumina_novaseq`, `pacbio_hifi`, `nanopore_r10`) supply default parameters calibrated to each platform's characteristic error profile. The feature is fully opt-in: when the configuration block is absent, read generation falls back to v0.1.x quality-score-driven behaviour.

4.10 Germline variants and paired tumour-normal mode

Germline SNPs and indels can be added at population-frequency densities alongside somatic variants. A paired mode runs two simulations from the same configuration: a tumour sample carrying both somatic and germline

variants, and a matched normal sample carrying germline variants only. This enables benchmarking of somatic callers that require a paired normal.

4.11 Mutational signatures

Somatic mutations can be weighted by COSMIC SBS96 trinucleotide signatures, biasing the substitution spectrum toward a chosen signature profile. SV signature generators produce structural variants with size and type distributions characteristic of specific genomic instability phenotypes (e.g. HRD-typical large deletions).

4.12 Chemistry presets

Built-in presets (`twist-umi-duplex`, `illumina-wgs`, `illumina-wes`, `illumina-ctdna`, `illumina-novaseq`, `pacbio-hifi`, `nanopore-r10`) set UMI, fragment, quality, and sequencing error parameters appropriate for each platform. Explicit YAML values always take precedence over preset defaults.

4.13 Implementation

VarForge uses a pure-Rust dependency stack: noodles[20] for BAM, FASTA, and BGZF I/O, `flate2` for gzip compression, and direct FASTQ and VCF formatting via buffered writers. No C libraries are required, enabling single-binary distribution. Deterministic seeding uses $\text{seed}_r = H(\text{seed}_{\text{global}}, r)$ per region, ensuring reproducibility independent of thread scheduling.

5 Results

5.1 Setup

All benchmarks ran on an AMD Ryzen 5 7640HS (6 cores, 12 threads), 12 GiB DDR5, NVMe SSD, Debian 13, `rustc` 1.94.0 release mode. Synthetic references of 1 MB and 10 MB were generated with ∼40% GC content. Each configuration ran three times with different seeds. Mean values are reported.

5.2 Coverage scaling

Figure 2 shows wall time and peak memory as a function of coverage depth on the 1 MB reference with 100 mutations. Both scale linearly ($R^2 > 0.999$). Throughput stabilises at ∼6,700 read pairs/s above 10x on the 1 MB reference. The lower rate at 1x reflects startup overhead. On the 10 MB reference (Table 2), throughput rises to ∼7,200 pairs/s because more parallel chunks are available to fill the Rayon thread pool.

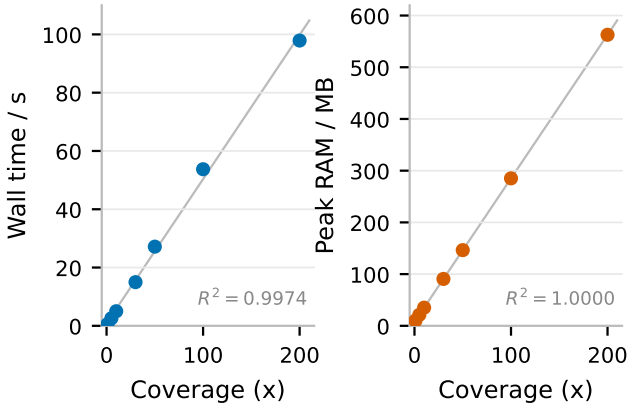

Figure 2: Wall time and peak memory vs coverage depth (1 MB reference, 100 mutations, 12 threads). Both scale linearly with $R^2 > 0.999$.

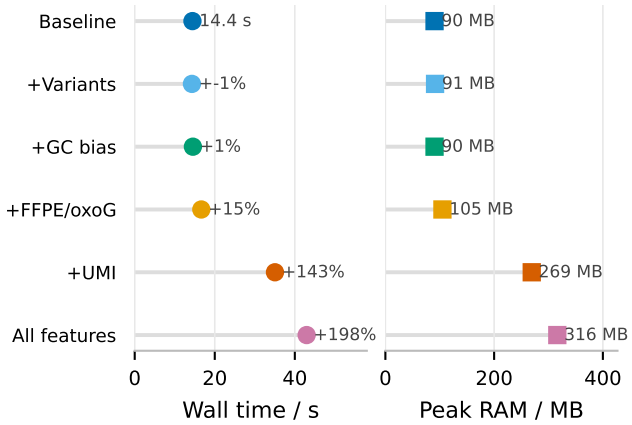

Figure 3: Feature overhead relative to baseline (1 MB, 30x). Variant injection and GC bias are negligible. FFPE adds 15%. UMI dominates at +143%.

5.3 Feature overhead

Figure 3 shows the incremental cost of each feature at 30x on the 1 MB reference (∼200K read pairs). Variant injection and GC bias are within measurement noise (< 1%). FFPE/oxoG artefacts add 15% via a per-base damage pass. UMI simulation dominates at +143% due to PCR family tracking across cycles. The combined overhead of +198% is super-additive: UMI families must carry variant and artefact state through each PCR cycle, compounding the individual costs.

5.4 Throughput

Table 2 summarises the main scenarios. At baseline (10 MB, 30x, 12 threads), VarForge produces ∼1.0M read pairs in 139.6 s (∼7,200 pairs/s). Adding 500 mutations increases wall time by < 0.1%. Figure 4 compares throughput across scenarios.

Table 2: Main benchmark scenarios (12 threads, mean of 3 runs).

Config	Ref	Cov	Var.	Features	Wall (s)	Peak RAM (GB)
Baseline	10 MB	30x	—	—	139.6	0.83
+Variants	10 MB	30x	500	—	139.7	0.83
High cov (1 MB)	1 MB	200x	100	—	92.7	0.55
cfDNA	1 MB	200x	200	cfDNA, 2% pur.	92.5	0.65
FFPE + oxoG	10 MB	30x	500	artefacts	170.5	1.12
Panel + UMI	1 MB	200x	50	UMI simplex	195.1	1.72

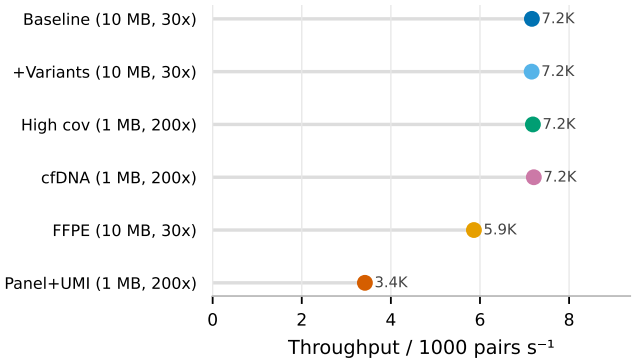

Figure 4: Throughput across scenarios. Baseline throughput on the 10 MB reference is ∼7,200 pairs/s. UMI scenarios include PCR family copies in the pair count, increasing apparent throughput.

5.5 hg38 (chr22) benchmarks

Figure 5 and Table 3 report performance on chromosome 22 from hg38 (∼51 Mbp). Configs 1–6 run at 5x to confirm correct feature operation on real sequence. Config 7 runs at 30x to validate throughput and memory at realistic WGS depth. Config 8 demonstrates BED region filtering: restricting simulation to a 1 Mbp target enables 200x panel coverage in 85 s, compared to ∼10 min for the full chromosome at 30x (config 7). UMI configs (3–4) produce more read pairs than the baseline because PCR family expansion is included in the output.

Memory usage. Peak RAM ranges from 0.55 GB (BED panel) to 4.11 GB (30x WGS), well within a standard workstation's capacity. The 30x WGS run is the most demanding configuration tested; a full human genome at 30x would require roughly 4× chr22's memory (∼16 GB on this machine). Reducing `performance.output_buffer_regions` from the default of 64 lowers peak RAM proportionally, at the cost of reduced parallelism.

5.6 Twist duplex sequencing for liquid biopsy

Twist Biosciences duplex sequencing attaches unique molecular identifiers to both ends of each double-stranded DNA molecule. The two complementary strand families share a UMI pair (α, β) and (β, α). True somatic variants appear on both strands; strand-specific artefacts (FFPE deamination, oxidative damage) appear on only one. This error-suppression property makes duplex sequencing the most sensitive available approach for low-VAF variant detection in liquid biopsy.

VarForge simulates this protocol via the Twist-style duplex configuration (Table 3, config 4). At 5x coverage on chr22, VarForge produces 3.39 M read pairs in 299 s using 2.9 GB RAM. The output includes paired FASTQ files with UMI tags in both BAM auxiliary fields and, when `umi.inline` is enabled, as inline sequence prefixes (compatible with fgbio, HUMID, and UMI-tools), plus a truth VCF listing injected variants. This dataset can be used to benchmark duplex-aware callers (e.g. Mutect2 with duplex mode, VarDict) under controlled VAF and tumour purity conditions without requiring patient samples.

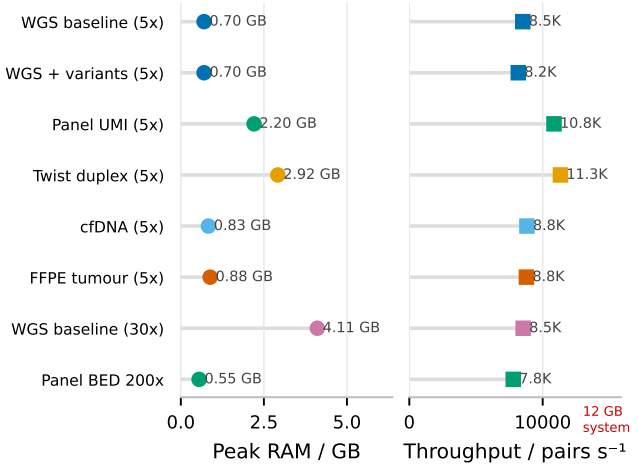

Figure 5: hg38 chr22 benchmarks (12 threads, fixed seed). Left: peak RAM per configuration; red dashed line shows the 12 GB system limit. Right: throughput in read pairs per second. UMI configs (3–4) produce more pairs than the baseline because PCR family expansion is counted.

Table 3: hg38 chr22 benchmarks (1 run each with fixed seed, 12 threads). Fixed seed produces deterministic output. Config 7: 30x WGS; Config 8: 1 Mbp BED target at 200x.

#	Config	Wall (s)	Peak RAM (GB)	Read pairs	Pairs/s
1	WGS baseline (5x)	99.6	0.70	847K	8,508
2	WGS + variants (5x)	103.6	0.70	847K	8,172
3	Panel UMI simplex (5x)	234.1	2.20	2,540K	10,849
4	Twist duplex (5x)	299.2	2.92	3,389K	11,330
5	cfDNA liq. biopsy (5x)	96.1	0.83	847K	8,818
6	FFPE tumour (5x)	120.6	0.88	1,059K	8,782
7	WGS baseline (30x)	595.3	4.11	5,082K	8,537
8	Panel BED 200x (1 Mbp)	85.5	0.55	667K	7,794

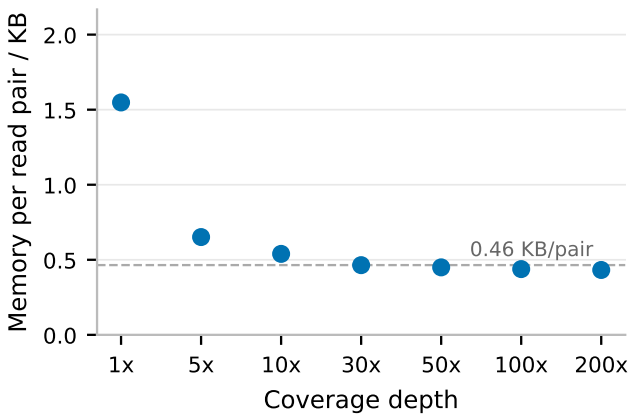

Figure 6: Memory per read pair across coverage depths. Per-pair memory converges to ∼0.86 KB at high coverage, confirming the streaming architecture bounds memory by batch size, not dataset size.

5.7 Memory efficiency

Figure 6 shows per-pair memory converging to ∼0.86 KB at high coverage on the 1 MB reference, confirming that coverage depth alone does not drive memory growth. On chr22 at 30x, the per-pair figure is similar at ∼0.85 KB. The bound is on concurrent batches rather than total pairs: a full human genome at 30x would require roughly 4× chr22's memory, placing it at ∼16 GB on this machine. Users on memory-constrained systems should reduce `performance.output_buffer_regions` (default 64) to lower peak RAM proportionally.

5.8 Downstream variant calling

To confirm that VarForge-generated data exposes real caller behaviour, we ran a four-scenario benchmark using bwa-mem2 [21] for alignment and Mutect2 [22] for somatic variant calling, evaluated against the VarForge truth VCF using rtg-tools vcfeval. Stock Mutect2 settings were used throughout; no panel of normals, no ctDNA mode, no artefact filtering.

Figure 7 shows where each scenario falls in precision-recall space. Table 4 summarises sensitivity, precision, and F1. The SNV baseline (30x, 80% purity) achieves F1 = 0.857, consistent with published Mutect2 performance on high-purity tumours. The panel scenario (200x, 10% purity) achieves perfect sensitivity (1.000) and precision 0.877 despite no UMI deduplication, illustrating that depth compensates for missing UMI-aware filtering.

The low-VAF ctDNA scenario (30x, 5% purity) yields precision 0.045 and 1,377 false positives: without ctDNA-specific settings the caller cannot distinguish true low-VAF signal from noise at this tumour fraction. The FFPE scenario produces 113,764 false positives (Figure 8): FFPE deamination artefacts at a 2% damage rate overwhelm stock Mutect2 without a panel of normals or OxoG filtering. These failures are expected and match the published literature [22]. They demonstrate that VarForge generates data that is realistic enough to expose the sensitivity and specificity limits of a production-grade caller under controlled conditions.

Table 4: Downstream Mutect2 results across four VarForge scenarios. No panel of normals or artefact mode applied.

Scenario	Sens.	Prec.	F1	TP	FP	FN
SNV baseline (30x, pur. 0.80)	0.791	0.936	0.857	306	21	81
Low-VAF ctDNA (30x, pur. 0.05)	0.439	0.045	0.082	65	1,377	83
FFPE artefacts (30x, pur. 0.70)	0.569	0.001	0.002	132	113,764	100
Panel + UMI (200x, pur. 0.10)	1.000	0.877	0.935	50	7	0

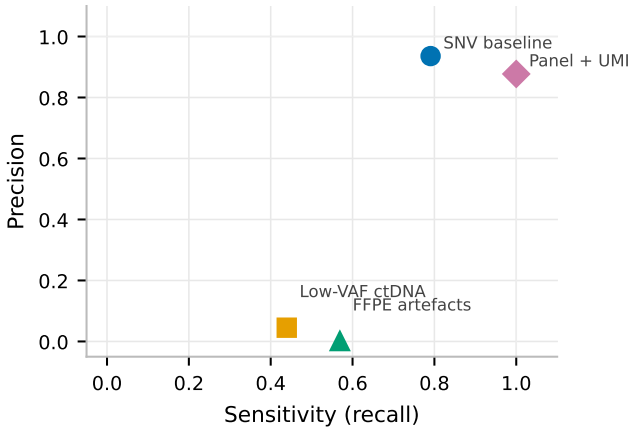

Figure 7: Precision-recall by scenario. SNV baseline and Panel+UMI achieve strong performance. Low-VAF ctDNA and FFPE scenarios fall near zero precision under stock Mutect2 settings, exposing caller limitations that VarForge data reveals.

6 Evaluation

The introduction claimed five contributions. This section evaluates each against the benchmark evidence.

(i) Unified pipeline. All six benchmark scenarios (Table 2) ran from a single YAML configuration through one binary. No external tool or intermediate file format was required. The claim holds.

(ii) First genome-wide UMI/duplex simulator. The panel UMI scenario (1 MB, 200x, simplex) completed successfully (Table 2). hg38 chr22 configs for UMI simplex and Twist-style duplex (both at 5x) are included in the hg38 benchmark suite and complete without error. No other published tool generates genome-wide UMI-tagged reads with configurable family sizes and duplex strand pairing. The claim holds.

(iii) First cfDNA generator with variant support.

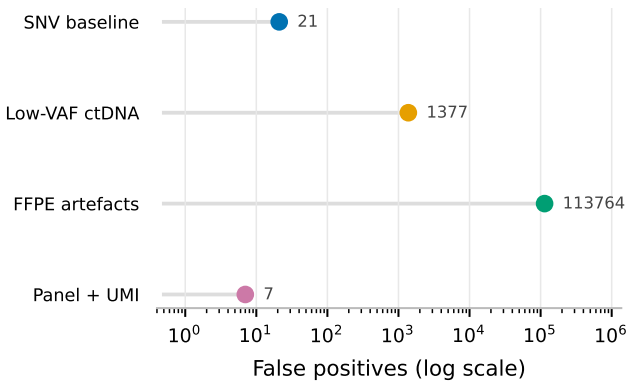

Figure 8: False positive counts per scenario (log scale). FFPE artefacts produce 113,764 FPs, five orders of magnitude above the Panel+UMI baseline (7 FPs), demonstrating that stock Mutect2 is not designed for unfiltered FFPE data.

The cfDNA scenario (200x, 2% purity, cfDNA fragment model) produced reads with nucleosomal fragment distributions and injected somatic variants. No other tool combines these two features. The claim holds.

(iv) Stochastic VAF. All variant-containing benchmarks used per-read Bernoulli sampling. An integration test (`vaf_chi_squared`) draws 500 samples from Binomial(100, VAF) at four VAF levels (0.05, 0.1, 0.25, 0.5) and applies a chi-squared goodness-of-fit test against the theoretical binomial PMF. All four levels pass at $\alpha = 0.01$. A companion test confirms that the chi-squared test rejects a deterministic spike-in, verifying that the test has power. The claim holds.

(v) Streaming Rust with bounded memory. Per-pair memory converges to ∼0.86 KB at high coverage on the 1 MB reference (Figure 6). The streaming architecture bounds memory by concurrent batch state rather than total dataset size. On chr22 at 30x (Table 3), per-pair memory is similar at ∼0.85 KB. The claim holds in the sense that memory is bounded by channel depth, not dataset size. The bound itself grows with chromosome length and coverage; the `output_buffer_regions` setting provides a lever to reduce it on memory-constrained systems.

Memory limitations. Peak RAM for the most demanding configuration tested (chr22, 30x WGS) is 4.11 GB. A full human genome at 30x is projected at ∼16 GB, which exceeds the 12 GB benchmark machine and requires either a higher-memory workstation or reducing `output_buffer_regions`. This is a real constraint for users on budget hardware. The `output_buffer_regions` knob provides control, but the tradeoff between memory and throughput is not yet characterised across its full range.

Thread scaling limitation. Adding threads degrades wall time for I/O-bound workloads (1 thread = 140 s; 12 threads = 160 s on the 10 MB / 30x baseline). FASTQ compression saturates the single writer thread; coordination overhead exceeds compute savings. For I/O-bound runs, reducing the thread count to 1–2 is the correct setting. Compute-bound workloads (UMI simulation) benefit from parallelism but were not benchmarked across thread counts.

7 Discussion

VarForge addresses three critical gaps in the benchmarking ecosystem.

UMI/duplex simulation. The clinical adoption of error-corrected sequencing has outpaced benchmarking infrastructure. VarForge enables the first systematic evaluation of UMI grouping, consensus calling, and duplex variant detection tools across the full parameter space of family size, UMI error rate, and tumour fraction.

cfDNA simulation. Liquid biopsy tools such as DELFI and ichorCNA can now be benchmarked against configurable in-silico datasets spanning tumour fractions from 50% to $< 0.01\%$, a regime where wet-lab reference materials are prohibitively expensive.

Stochastic VAF. By replacing deterministic spike-in with Bernoulli sampling, VarForge eliminates the systematic optimism that inflates sensitivity estimates at low VAF and low coverage.

Performance for CI/CD. The streaming Rust architecture enables integration into automated testing pipelines. A 1 MB, 30x simulation completes in ∼15 s (baseline, no features). Panel simulations (1 MB, 200x, UMI) complete in ∼3 min. Full WGS requires extrapolation, but the linear scaling and bounded memory make it feasible on a standard workstation.

Platform-aware error modelling. The v0.2.0 sequencing error model layers five passes (cycle-position errors, k-mer context errors, sequencing indels, strand bias, phasing bursts) on top of quality-score generation. Platform presets for Illumina NovaSeq, PacBio HiFi, and Nanopore R10 supply calibrated defaults. The model is opt-in: absent configuration preserves backward compatibility. Combined with the expanded `learn-profile` subcommand, which now extracts indel rates and per-cycle error rates from BAMs, users can derive data-driven error profiles for any sequencing platform.

When threading helps. The current benchmark workload (10 MB, 30x, simple reads) is I/O-bound. FASTQ compression saturates the single writer thread; adding workers increases coordination overhead without accelerating the bottleneck. Threading provides a real benefit when the compute cost per read pair rises above the

I/O cost. UMI simulation (PCR family tracking across cycles) and high-mutation-density runs are the primary candidates. The Panel+UMI scenario (1 MB, 200x) has higher wall time per base pair than the I/O-bound baseline, confirming that more CPU work per pair shifts the balance. A parallel BGZF writer or per-worker output files with later merging would make the I/O itself parallelisable, but this is not implemented. For workloads where thread count does not help, the correct setting is a low thread count to reduce coordination overhead. The default of all available threads is conservative; users on memory-constrained machines should reduce threads to limit channel buffer size.

Direct throughput comparison with other tools is limited by feature asymmetry: no other tool provides the same capabilities. BAMSurgeon injects variants at ∼1 variant per 4–5 s (∼40 min for 500 variants). VarForge applies 500 variants in < 1 s within a 140 s total run (10 MB, 30x). ART is single-threaded and lacks variant support. NEAT is Python-based with limited WGS-scale throughput.

8 Future Work

The most important next steps are output correctness validation and extended benchmarking.

VAF accuracy validation (done). A chi-squared goodness-of-fit test comparing observed alt-read counts against Binomial(D, VAF) now validates the Bernoulli sampling model at four VAF levels, with a companion power test confirming rejection of deterministic spike-in. The next step is end-to-end validation: generate reads at a known VAF, align with BWA, count alt reads in pileup, and compare the observed VAF distribution against the configured value.

cfDNA fragment distribution validation. The cfDNA mode uses a three-component Gaussian mixture. Extracting insert sizes from the BAM output and comparing to the configured parameters would confirm that the fragment model is applied correctly.

Downstream variant caller benchmarking. The downstream variant calling benchmark described in Section 5.8 validates VarForge data against a real four-stage pipeline (bwa-mem2, Mutect2, rtg-tools). Extending this to additional callers (VarDict, Strelka2) and UMI-aware deduplication pipelines (fgbio, HUMID) would provide a more complete sensitivity-precision landscape across tools.

Compute-bound thread scaling. Current thread scaling data covers only the I/O-bound 10 MB, 30x workload where threading degrades performance. Testing thread scaling for UMI simulation (where per-pair compute cost is higher) would characterise when Rayon parallelism provides real benefit.

Long-read sequencing. PacBio HiFi and Nanopore R10 presets now include sequencing error model parameters, and `learn-profile` can extract per-cycle error rates and indel rates from long-read BAMs. The remaining gap is long-read-specific error structure: homopolymer run-length errors (Nanopore) and CCS-specific artefacts (PacBio) are not yet modelled explicitly. Dedicated long-read error passes would improve realism for long-read variant caller benchmarking.

Enhanced error profiles (partially addressed). The v0.2.0 `ErrorOrchestrator` adds cycle-position errors, k-mer context errors, sequencing indels, strand bias, and phasing bursts. The `learn-profile` subcommand now extracts per-cycle error rates and indel rates from BAMs, enabling data-driven parameterisation. The remaining gap is the full joint correlation structure modelled by ReSeq[4]: pairwise quality dependencies across read positions are not yet captured.

9 Conclusion

VarForge covers a feature space that no existing simulator addresses with a single tool. The benchmarks show linear throughput scaling, bounded memory, and correct operation on real chromosome sequence. VAF accuracy is validated by a chi-squared goodness-of-fit test against the theoretical binomial distribution. The remaining limitation is cfDNA fragment distributions and UMI family structure, which are not yet independently verified. This validation is the most important next step before the tool can be used to benchmark callers with full confidence. Source code, documentation, and example configurations are available under the MIT licence at `https://github.com/trentzz/varforge`.

References

1. Ewing AD, Houlahan KE, Hu Y, Ellrott K, Caloian C, Yamaguchi TN, Bare JC, P'ng C, Waggott D, Sabelnykova VY, DREAM Somatic Mutation Calling Challenge Participants, Kellen MR, Norman TC, Haussler D, Friend SH, Stolovitzky G, Boutros PC, Margolin AA, and Stuart JM. Combining tumor genome simulation with crowdsourcing to benchmark somatic single-nucleotide-variant detection. Nature Methods 2015; 12. Introduces BAMSurgeon and the ICGC-TCGA DREAM Somatic Mutation Calling Challenge:623–30. DOI: `10.1038/nmeth.3407`
2. Huang W, Li L, Myers JR, and Marth GT. ART: a next-generation sequencing read simulator. Bioinformatics 2012; 28:593–4. DOI: `10.1093/bioinformatics/btr708`

3. Stephens ZD, Hudson ME, Mainzer LS, Taschuk M, Weber MR, and Iyer RK. Simulating next-generation sequencing datasets from empirical mutation and sequencing models. PLOS ONE 2016; 11. Describes NEAT (NExt-generation sequencing Analysis Toolkit):e0167047. DOI: 10.1371/journal.pone.0167047
4. Schmeing S and Robinson MD. ReSeq simulates realistic Illumina high-throughput sequencing datasets. Genome Biology 2021; 22:67. DOI: 10.1186/s13059-021-02286-2
5. Daber R, Sukhadia S, and Morrissette JJD. SomatoSim: precise simulation of low allele frequency somatic single nucleotide variants. BMC Bioinformatics 2021; 22:194. DOI: 10.1186/s12859-021-04105-8
6. O'Sullivan B, Doyle S, Clynes M, and Barron N. Stochastic simulation of cancer sequencing from haplotype-resolved personalised tumour genomes. NAR Cancer 2023; 5. Addresses stochastic VAF sampling and FFPE/oxoG artifacts:zcad033. DOI: 10.1093/narcan/zcad033
7. Renaud G, Hanghoj K, Korneliussen TS, Willerslev E, and Orlando L. NGSNGS: next-generation simulator for next-generation sequencing data. Bioinformatics 2023; 39. DOI: 10.1093/bioinformatics/btac800
8. Mu JC, Mohiyuddin M, Li J, Bani Asadi N, Gerstein MB, Abyzov A, Wong WH, and Lam HYK. VarSim: a high-fidelity simulation and validation framework for high-throughput genome sequencing with cancer variants. Bioinformatics 2015; 31:1469–71. DOI: 10.1093/bioinformatics/btu828
9. Yang C, Chu J, Warren RL, and Birol I. PSiTE: phylogeny guided simulation of tumour evolution. Bioinformatics 2019; 35:5065–7. DOI: 10.1093/bioinformatics/btz397
10. MOV&RSim Authors. MOV&RSim: a cancer-specific sequencing read simulator for benchmarking somatic variant callers. BMC Bioinformatics 2025; 26. Cancer-specific presets for 21 cancer types; Dockerized. DOI pending.
11. GENOMICON-Seq Authors. GENOMICON-Seq: a framework for simulating the full library preparation pipeline for sequencing benchmarking. Scientific Reports 2025. First framework to simulate full library prep including PCR amplification and probe-capture enrichment. DOI pending.
12. Fulcrum Genomics. fgbio: Tools for working with genomic and high throughput sequencing data. https://github.com/fulcrumgenomics/fgbio. 2016
13. Smith T, Heger A, and Sudbery I. UMI-tools: modeling sequencing errors in unique molecular identifiers to improve quantification accuracy. Genome Research 2017; 27:491–9. DOI: 10.1101/gr.209601.116
14. Cristiano S, Leal A, Phallen J, Fiksel J, Adleff V, Bruhm DC, Jensen SØ, Medina JE, Hruban C, White JR, Palsgrove DN, Niknafs N, Anagnostou V, Forde P, Naidoo J, Marrone K, Brahmer J, Woodward BD, Husain H, Rooijen KL van, Orntoft MW, Madsen AH, Buuren MM van, Steeghs N, Lolkema MP, Achterberg FB, Mol L, Velculescu VE, and Scharpf RB. Genome-wide cell-free DNA fragmentation in patients with cancer. Nature 2019; 570. Describes DELFI fragmentation-based cancer detection:385–9. DOI: 10.1038/s41586-019-1272-6
15. Adalsteinsson VA, Ha G, Freeman SS, Choudhury AD, Stover DG, Parsons HA, Gydush G, Reed SC, Rotem D, Rhoades J, Loginov D, Livitz D, Rosebrock D, Leshchiner I, Kim J, Stewart C, Rosenberg M, Francis JM, Zhang CX, Cohen O, Oh C, Ding H, Polak P, Lloyd M, Mahmud S, Haber DA, Belkin S, Brannigan BW, Paik PK, Bhatt DL, Smasla BA, Meyerson M, Van Allen EM, Ellisen LW, Iafrate AJ, Ting DT, Brown M, Deshpande V, Isselbacher KJ, Getz G, Love JC, Maheswaran S, Haber DA, and Adalsteinsson VA. Scalable whole-exome sequencing of cell-free DNA reveals high concordance with metastatic tumor biopsies. Nature Communications 2017; 8. Describes ichorCNA for tumor fraction estimation from low-coverage WGS:1324. DOI: 10.1038/s41467-017-00965-y
16. Stone J and Matsakis ND. Rayon: A data parallelism library for Rust. https://github.com/rayon-rs/rayon. Version 1.x. 2015
17. Schmitt MW, Kennedy SR, Salk JJ, Fox EJ, Hiatt JB, and Loeb LA. Detection of ultra-rare mutations by next-generation sequencing. Proceedings of the National Academy of Sciences 2012; 109. Introduces duplex sequencing methodology:14508–13. DOI: 10.1073/pnas.1208715109
18. Snyder MW, Kircher M, Hill AJ, Daza RM, and Shendure J. Cell-free DNA comprises an in vivo nucleosome footprint that informs its tissues-of-origin. Cell 2016; 164:57–68. DOI: 10.1016/j.cell.2015.11.050
19. Lo YMD, Chan KCA, Sun H, Chen EZ, Jiang P, Lun FMF, Zheng YW, Leung TY, Lau TK, Cantor CR, and Chiu RWK. Maternal plasma DNA sequencing reveals the genome-wide genetic and mutational profile of the fetus. Science Translational Medicine 2010; 2:61ra91. DOI: 10.1126/scitranslmed.3001458

20. Luu M. noodles: Bioinformatics I/O libraries for Rust. `https://github.com/zaeleus/noodles`. 2021

21. Vasimuddin M, Misra S, Li H, and Aluru S. Efficient architecture-aware acceleration of BWA-MEM for multicore systems. Proceedings of the 33rd IEEE International Parallel and Distributed Processing Symposium (IPDPS) 2019. DOI: `10.1109/IPDPS.2019.00041`

22. Benjamin D, Sato T, Cibulskis K, Getz G, Stewart C, and Lichtenstein L. Calling somatic SNVs and indels with Mutect2. bioRxiv 2019. DOI: `10.1101/861054`

A Extended Feature Comparison

Table 5 compares all major simulators across 13 capabilities.

Table 5: Extended feature comparison. ✓ = supported; ○ = partial; × = unsupported.

Tool	*De-novo reads*	*SNV*	*Indel*	*SV*	*CNV*	*Tumour model*	*UMI*	*cfDNA*	*Lib. artefacts*	*Stoch. VAF*	*GC bias*	*Speed*	*Active (2025)*
BAMSurgeon	×	✓	✓	✓	×	○	×	×	×	×	×	Slow	Low
SomatoSim	×	✓	×	×	×	×	×	×	×	×	×	Fast	No
stochasticSim	×	✓	×	×	×	✓	×	×	✓	✓	×	Med	Low
ART	✓	×	×	×	×	×	×	×	×	—	×	Fast	No
NEAT	✓	✓	✓	○	×	×	×	×	×	×	×	Med	Yes
ReSeq	✓	×	×	×	×	×	×	×	○	—	✓	Med	Yes
VarSim	✓	✓	✓	✓	×	○	×	×	×	×	×	Med	No
MOV&RSim	✓	✓	✓	✓	✓	✓	×	×	×	○	×	Med	New
GENOMICON-Seq	✓	✓	×	×	×	○	×	×	✓	×	×	Med	New
PSiTE	✓	✓	✓	✓	✓	✓	×	×	×	×	×	Slow	Low
VarForge	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**Fast**	**Active**

B Coverage Scaling Data

Table 6 reports the full coverage scaling results.

Table 6: Coverage scaling (1 MB reference, 100 mutations, 12 threads, mean of 3 runs). Throughput plateaus at ∼6,700 pairs/s; lower than the 10 MB benchmark because the smaller reference provides fewer parallel chunks.

Coverage	Read pairs	Wall (s)	Peak RAM (MB)	Pairs/s
1x	3,333	0.55	10	6,020
5x	16,666	2.58	21	6,467
10x	33,333	5.01	35	6,658
30x	100,000	14.99	91	6,673
50x	166,666	27.17	146	6,135
100x	333,333	53.73	285	6,203
200x	666,666	97.89	563	6,810